

SAHAKAR SETU — Voice AI (Detailed)

How speech becomes text, how the answer becomes speech, and why the service never goes silent.

1. Two Speech Engines, One Brain

Sahakar Setu runs two independent speech stacks that share the same RAG + LLM core:

Path	STT	TTS	Where
IVR phone call (feature phone, no internet)	Vapi.ai <code>talkscribe</code> (Whisper, <code>language: hi</code>)	Vapi voice <code>Naina</code> (<code>language: hi</code>)	Cloud — any phone on the planet calls +1 346 998 6840
Kiosk / PWA voice (on-device mic)	Local Vosk <code>vosk-small-hi-0.22</code> via persistent Python worker	<code>edge-tts</code> (Hindi neural voice) via persistent Python worker	On-premise — works on the kiosk even on slow links

Vapi is used with **key-less providers only** (`talkscribe` STT + Vapi TTS) — zero paid provider credentials, keeping the demo cost at ₹0 while staying fully live.

2. IVR Call Flow (Vapi custom-LLM integration)

The server exposes an OpenAI-compatible endpoint that Vapi's assistant calls every turn: `POST /webhook/vapi/chat/completions` (`handleVAPICustomLLM` in `src/channels/ivr/index.ts`).

- Vapi transcribes the caller's Hindi speech (`talkscribe`/`Whisper`) and sends an OpenAI-format `messages` array to our endpoint.
- Endpoint supports both `stream: true` (SSE chunks — `data: {"choices":[{"delta":...}]}` ... `data: [DONE]`) and plain JSON.
- The endpoint authenticates the request with `VAPI_LLM_SECRET` passed as `?key=` in the assistant's `model.url` (401 otherwise).
- Call identity: the assistant's system message carries `CallRef: {{call.id}}` ; the server maps the call to a session via `sessions.external_ref = 'vapi:<callId>'` — so every call is one persistent case record.
- Same pipeline as web: language detect → retrieve (top-5, BM25) → `generateAnswer` with the voice prompt (20b model first, temp 0.7, ≤4 history turns, 250-char passages) → answer returned to Vapi → Naina speaks it.
- End of call: `/webhook/vapi/call` receives the report; if the session somehow has 0 turns, `artifact.messages` is backfilled (dedupe guard) and the session is marked ended.

```
// Voice prompt rules (groq.ts) — a LIVE phone call, not a FAQ document
// 1. React to the caller's exact words; never repeat phrasing across turns
// 2. Greet only on the first turn; never greet again mid-call
```

```
// 3. Everyday spoken Hindi, warm ("जी", "चिंता मत कीजिए"); no jargon, no markdown
// 4. Ask ONE follow-up question when a detail is missing.
// 5. Empathy in your own words – vary it every time.
// 6. 2-4 short spoken sentences per turn.
// 7. FORBIDDEN on calls: "नीचे देखें", website/app/link mentions, bullets, tables
// 8. Out-of-domain → exact boundary message (verbatim).
```

3. Local Vosk STT Worker (kiosk / PWA)

The Node server spawns a long-lived Python worker (`scripts/vosk_worker.py`) that keeps the Vosk model loaded in memory — no reload per request. Node talks to it over a JSON-lines stdin/stdout protocol:

```
// Node side – src/modules/stt/vosk.ts
this.process = spawn('python', ['scripts/vosk_worker.py'], {
  env: { ...process.env, VOSK_MODEL_PATH: process.env.VOSK_MODEL_PATH || 'models' }
});
// requests: {"type": "transcribe", "id": "...", "path": "...wav", "lang": "hi"}
// responses: {"type": "ready"} | {"type": "transcript", "id": "...", "text": "...", "is_final": true} | {"type": "error", "id": "...", "text": "...", "is_final": true}
```

- **Model:** `vosk-small-hi-0.22` stored locally in `models/` — no cloud STT dependency on the kiosk.
- **Pre-warm:** workers are created at server boot (~5 s model load), so the first user request is fast.
- **Self-healing:** `isAlive()` checks; on worker exit all pending requests reject and the next call respawns the worker.
- **Windows fix:** the Python workers reconfigure stdin/stdout to UTF-8 (`sys.stdin.reconfigure(encoding='utf-8')`) — the cp1252 default silently corrupted Devanagari and blocked buffers (verified: `/tts` returns 53 KB MP3, `/stt` transcribes Hindi correctly).
- **Empty transcript guard:** if nothing was heard, the API politely re-asks — “Mujhe aapki awaaz sunai nahi di. Kripya dohrayein.”

4. TTS Fallback Chain — the service never goes silent

Tier	Provider	Notes
1	edge-tts (Microsoft neural Hindi voice) via persistent Python worker	Primary; MP3 output; in-memory LRU cache (200 entries) + Redis (24 h) so repeated answers don't re-synthesize
2	gTTS (Google Translate TTS)	Automatic fallback; used on Render where edge-tts is IP-blocked (403)
3	SMS	Last resort — the answer text is delivered as an SMS, so a voice question always gets an answer

5. Latency Budget (evaluation target: median voice turn < 2.5 s end-to-end)

Stage	Budget	How
-------	--------	-----

STT	~0.5 s	Vapi streaming / warm Vosk worker (no model reload)
Retrieval	~0.2 s	BM25 over ≤ 1000 in-memory-ish DB rows, single query
LLM	~1.2 s	20b model first on voice channels, 250-char passages, 350 max tokens
TTS	~0.4 s	Cache hit common; Vapi Naina streams the first words early

6. Verified Live (24 Sep 2026)

- Live call to **+1 346 998 6840**: Hindi greeting, natural conversation, cited answers spoken cleanly.
- Kiosk /voice: 53 KB Hindi MP3 from /tts; /stt returns correct transcript (worker UTF-8 fix verified).
- ngrok tunnel (ethanol-cash-peculiar.ngrok-free.dev) serving the Vapi webhook; kiosk “awaaz samajh nahi aayi” symptom == dead tunnel, restartable with the pinned domain.